Davante

18

BASE DE DATOS

Bases de datos objeto-relacionales

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Definición y características 5

/ 3. Tipos de datos objeto 6

/ 4. Definición de tipos de datos objeto 7

/ 5. Declaración de atributos 8

/ 6. Declaración de métodos 9

/ 7. Métodos estáticos 10

/ 8. Parámetro SELF 11

/ 9. Fundamentos de la sobrecarga de métodos 11

9.1. Condiciones y reglas para la sobrecarga 12

/ 10. Aplicación avanzada de la sobrecarga 12

10.1. Optimización y eficiencia 12

10.2. Diseño orientado al mantenimiento 13

/ 11. Constructores 13

/ 12. Caso práctico 1: “Creación de tipo de dato objeto en Oracle Express Database” 14

ÍNDICE

/ 13. Caso práctico 2: “Definición de atributos y métodos para números complejos” 15

/ 14. Resumen y resolución del caso práctico de la unidad 16

/ 15. Bibliografía 17

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer las características principales de las bases de datos objeto-relacionales y sus campos de aplicación.

Comprender el concepto de tipo de dato objeto y sus principales elementos, así como su definición.

Comprender y manejar los conceptos y declaración de atributos, métodos, sobrecarga y constructores.

Conocer y manejar el concepto de parámetro SELF adaptado a las bases de datos objeto-relacionales.

/ 1. Introducción y contextualización práctica

En este tema, nos adentraremos en el estudio de las bases de datos objeto-relacionales (BDOR), que aprovechan el paradigma orientado a objetos, similar a lo que ocurre en lenguajes de programación como Java o PL/SQL. La programación orientada a objetos está estrechamente vinculada con el desarrollo de aplicaciones complejas y reutilizables.

Exploraremos conceptos clave de la orientación a objetos, como tipos de datos objeto, sobrecarga, métodos, atributos y constructores. Además, examinaremos los distintos tipos de datos objeto y cómo se definen para representar elementos del mundo real.

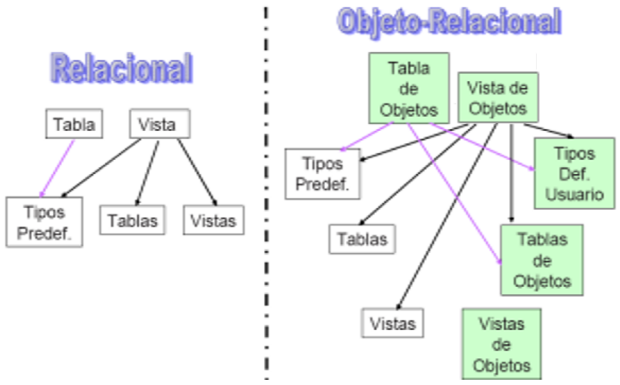


Fig.1. Elementos del modelo relacional vs. objeto-relacional.

Hoy en día, este enfoque de bases de datos está ganando importancia, pues facilita la reutilización de objetos para desarrollar aplicaciones más avanzadas y eficientes, integrándose con otras tecnologías y programas en diferentes entornos de programación. Oracle, en particular, ofrece un sólido soporte para diversos tipos de objetos, simplificando la labor de desarrolladores y programadores al incorporar este enfoque en su sistema de gestión de bases de datos.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio Intro. "Modelado del tipo de dato objeto coche"

https://on.soundcloud.com/53SARvGErt8G7hxi9

/ 2. Definición y características

Las bases de datos objeto-relacionales representan una evolución del modelo relacional, fusionando características de ambas tecnologías en una **base de datos híbrida** que **combina** elementos **relacionales** y de **objetos**. La idea central de este enfoque es que el usuario tenga la capacidad de **definir sus propios tipos de datos**.

Las principales **características** de la base de datos relacional frente a la base de datos objeto-relacional (BDOR) se resumen en la siguiente tabla:

Sistema	Características
BD relacional	Tipos de datos simples. Lenguajes de consulta robustos. Alta seguridad.
BDOR	Tipos de datos complejos. Lenguajes de consultas igualmente robustos. Integración con lenguajes de programación. Alto rendimiento y productividad. Mayor coste y complejidad.

Tabla 1. Características de la BD relacional vs. la BDOR.

En resumen, las BBDD objeto-relacionales van más allá de las BBDD relacionales tradicionales, permitiendo la implementación de sistemas más complejos. No obstante, las relacionales siguen siendo capaces de abordar una amplia variedad de escenarios.

En el caso de **Oracle**, la **integración de los modelos relacional y orientado a objetos** se realiza de manera **eficiente** y **clara**, dando como resultado un **sistema** extremadamente **potente**. La siguiente imagen muestra la **correspondencia entre los elementos** del modelo **orientado a objetos** y los que hemos estudiado en el modelo **relacional**:

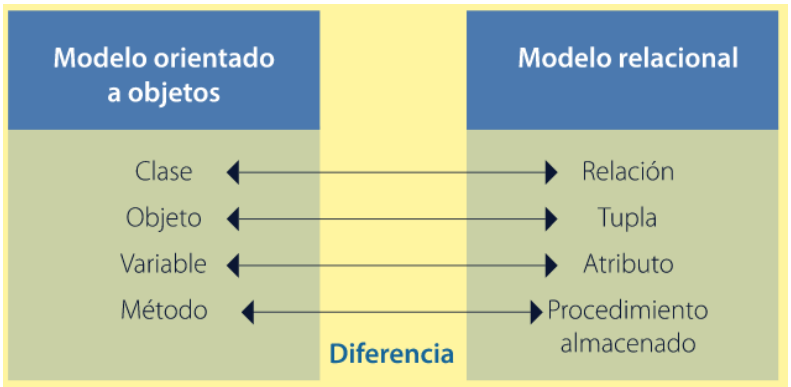


Fig.2. Elementos del modelo relacional vs. BDOR.

Escucha el siguiente audio para conocer más características de este tipo de bases de datos:

Audio 1. "Encapsulación, herencia y polimorfismo"
https://on.soundcloud.com/8KwFXAFMpgx3s1Rb6

/ 3. Tipos de datos objeto

Un tipo de dato objeto es **una representación creada por el usuario para reflejar un elemento del mundo real**, y se compone de varios elementos:

- **Nombre**: Identifica el tipo de dato objeto. Por ejemplo, un tipo de dato objeto podría ser un alumno.
- **Atributos**: Son las propiedades o características que definen al tipo de dato objeto. Estos atributos pueden ser tipos de datos básicos o definidos por el usuario. En el caso de un tipo de dato objeto alumno, algunos atributos podrían incluir el nombre, el DNI, el grupo, las calificaciones, entre otros.
- **Métodos**: Son procedimientos o funciones que se desarrollan en PL/SQL o en algún otro lenguaje de programación. Para un tipo de dato objeto alumno, algunos métodos útiles podrían ser asignar un curso a un alumno, modificar sus calificaciones, etc.

El tipo de dato objeto se estructura en dos partes:

- **Especificación de datos (pública)**: Contiene la información necesaria para utilizar los métodos del tipo de dato objeto.
- **Cuerpo (privado)**: Incluye los métodos, como se muestra en la siguiente imagen.

Investigamos...

Investiga utilizando Google Scholar sobre los principios básicos de la programación orientada a objetos para comprender los detalles de cada uno y las ventajas que ofrece a la hora de programar.

Al igual que en la programación orientada a objetos, la **manipulación** y **modificación** de los datos deben realizarse **exclusivamente a través de los métodos**. El cuerpo del tipo de dato objeto permanece privado, mientras que la especificación es pública, facilitando así la mejora o depuración de los métodos sin necesidad de modificar la especificación ni las aplicaciones cliente. Esto además facilita la reutilización del código en otras partes.

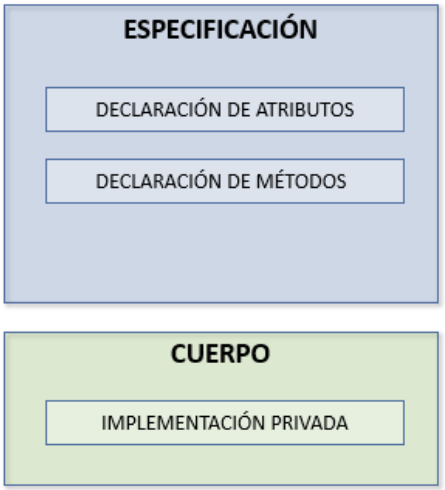


Fig.3. Estructura de un tipo de dato objeto.

Otro concepto importante es el de **instancia**, que **se refiere a una representación concreta de un tipo de dato objeto real con valores específicos en sus atributos**. Por ejemplo, el alumno Juan García Pérez sería una instancia de un tipo de dato objeto alumno, representando a un alumno real.

/ 4. Definición de tipos de datos objeto

Como vimos en el apartado anterior, **es necesario declarar todos los atributos antes de los métodos**. Además, **no se pueden declarar atributos dentro del cuerpo del tipo de dato objeto**. Sin embargo, es **posible crear tipos de datos objeto que no tengan métodos**, es decir, que **no cuenten con** un **cuerpo**.

Es importante recordar que **no se pueden declarar constantes, excepciones, cursores o tipos dentro de la especificación**. La **sintaxis** utilizada para **definir un tipo de dato objeto** en Oracle se muestra en la siguiente imagen:

```
CREATE TYPE nombre_tipo AS OBJECT (
    Declaración_de_atributos
    Declaración_de_métodos
) ;
```

Fig.4. Sintaxis de definición de un tipo de dato objeto en Oracle.

También es posible **reemplazar un tipo de dato objeto existente** utilizando la instrucción de la siguiente imagen:

```
CREATE OR REPLACE TYPE nombre_tipo AS OBJECT;
```

Fig.5. Sintaxis para reemplazar un tipo de dato objeto existente.

Para **eliminar un tipo de dato objeto existente**, se puede utilizar el comando **DROP**, como se muestra en la siguiente imagen:

```
DROP TYPE nombre_tipo;
```

Fig.6. Sintaxis para eliminar un tipo de dato objeto existente.

Las opciones para declarar atributos y métodos se analizarán con mayor detalle en los próximos apartados de este tema. A continuación, se muestra una imagen con un ejemplo de cómo se declaran dos tipos de datos objeto en Oracle, el tipo de dato objeto 'alumno' y el tipo de dato objeto 'profesor', con sus respectivos atributos:

```
CREATE TYPE alumno AS OBJECT (
    nombre VARCHAR2(50),
    apellidos VARCHAR2(100),
    dni VARCHAR2(9),
    calificación NUMBER
) ;

CREATE TYPE profesor AS OBJECT (
    nombre VARCHAR2(50),
    apellidos VARCHAR2(100),
    dni VARCHAR2(9),
    asignatura VARCHAR2(50)
) ;
```

Fig.7. Ejemplo de declaración de los tipos de datos objeto alumno y profesor.

Vídeo 1. "Creación de objetos en Oracle Express Database"

https://bit.ly/2Luw80o

/ 5. Declaración de atributos

Un atributo es un elemento que se asemeja a una variable. En su declaración, se asigna un nombre único dentro del objeto y se define un tipo de dato asociado.

Este tipo de dato puede ser cualquiera de los soportados por Oracle, con excepción de los siguientes:

- LONG y LONG RAW.
- NCHAR, NCLOB y NVARCHAR2.
- MLSLABEL y ROWID.
- Tipos específicos de PL/SQL: BINARY_INTEGER, BOOLEAN, %TYPE y %ROWTYPE.
- Tipos que hayan sido definidos en el interior de un paquete de PL/SQL.

Además, no está permitida la asignación de la restricción **NOT NULL**.

En la siguiente imagen se muestra un ejemplo de tipo de dato objeto y sus atributos:

```
CREATE TYPE direccion AS OBJECT (
    calle VARCHAR2(50),
    ciudad VARCHAR2(20),
    código_postal number(5)
) ;
```

Fig.8. Ejemplo de declaración de tipo de dato objeto y atributos.

Una gran ventaja es que **el tipo de dato de un atributo puede ser otro tipo de dato objeto**. En la siguiente imagen se muestra un ejemplo:

```
CREATE TYPE profesor AS OBJECT (
    nombre VARCHAR2(50),
    apellidos VARCHAR2(100),
    dni VARCHAR2(9),
    asignatura VARCHAR2(50),
    domicilio direccion
) ;
```

Fig.9. Ejemplo de atributo como tipo de dato objeto.

Con el comando **ALTER TYPE** se puede modificar la estructura de un tipo de dato objeto ya creado modificando, añadiendo o eliminando sus atributos. Con la instrucción **MODIFY ATTRIBUTE** es posible modificar atributos existentes; la instrucción **ADD ATTRIBUTE** permite añadir nuevos atributos, y la instrucción **DROP ATTRIBUTE** se utiliza para eliminar atributos. Esto se ejemplifica en la siguiente imagen:

```
ALTER TYPE profesor DROP ATTRIBUTE dni;
ALTER TYPE profesor ADD ATTRIBUTE edad NUMBER(2),
                    MODIFY ATTRIBUTE apellidos VARCHAR2(150);
```

Fig.10. Ejemplos de diferentes acciones sobre atributos.

/ 6. Declaración de métodos

Los métodos (de instancia) **se declaran usando** la palabra clave **MEMBER**, seguida de alguna de las siguientes opciones para especificar el tipo de método:

- **FUNCTION**: Se utiliza cuando el método es una función, lo que implica que debe retornar un valor al ser ejecutado.
- **PROCEDURE**: Se emplea cuando el método es un procedimiento, el cual no devuelve un valor al finalizar.

Un método **no debe tener el mismo nombre que el tipo de dato objeto o alguno de sus atributos**. Sin embargo, **existe una excepción** en la que un método puede llevar el **mismo nombre que el tipo de dato objeto**. En estos casos, el método **recibe un tratamiento especial**, el cual explicaremos en detalle más adelante.

La declaración del método incluye el nombre del método, junto con **sus parámetros** (si los hay) **y el tipo de dato devuelto** en caso de que sea una función. **El código del método** se implementa en **el cuerpo**, que es **la parte que se ejecuta** para realizar las acciones deseadas.

En la siguiente imagen, se muestra un ejemplo de cómo declarar dos métodos dentro del tipo de dato objeto 'profesor':

```
CREATE TYPE profesor AS OBJECT (
    nombre VARCHAR2(50),
    asignatura VARCHAR2(50),
    fecha_nac DATE,
    salario NUMBER,
    MEMBER FUNCTION edad RETURN NUMBER,
    MEMBER PROCEDURE aumentoSalario(a NUMBER)
) ;
```

Fig.11. Ejemplo de declaración de métodos.

Para cada **declaración de método** indicada en el **bloque de especificación** del tipo de dato objeto, debe incluirse su **implementación** en el bloque **cuerpo**. Para el tipo de dato objeto anterior podríamos tener el cuerpo mostrado en la siguiente imagen, donde se implementan los métodos declarados antes:

```
CREATE TYPE BODY profesor AS
    MEMBER PROCEDURE aumentoSalario (a NUMBER) IS BEGIN
    salario := salario + a;
    END aumentoSalario;
    MEMBER FUNCTION edad RETURN NUMBER IS BEGIN
        …
    END edad;
END;
```

Fig.12. Ejemplo de implementación de los métodos en el cuerpo de un tipo de dato objeto.

Vídeo 2. "Definición de métodos en Oracle Express Database"

https://bit.ly/2XHevNz

/ 7. Métodos estáticos

Aunque menos comunes, los métodos estáticos (o de clase) se pueden definir utilizando la palabra clave **STATIC**. Estos métodos **no pueden acceder a los atributos de instancia del objeto**, pero pueden ser útiles para operaciones que son generales y no dependen de una instancia específica del objeto.

En la siguiente imagen se muestra un ejemplo de la definición de un método estático en un tipo de dato objeto:

```
CREATE TYPE Persona AS OBJECT (
    nombre VARCHAR2(50),
    edad NUMBER,
    STATIC FUNCTION edad_jubilacion
           RETURN NUMBER
);
/

CREATE TYPE BODY Persona AS
    STATIC FUNCTION edad_jubilacion
           RETURN NUMBER IS
    BEGIN
        RETURN 65;
    END edad_jubilacion;
END;
```

Fig.13. Creación de métodos estáticos.

> **Investigamos...**
>
> Utiliza Google Scholar u otra plataforma para investigar acerca de la diferencia entre los métodos de instancia y los métodos estáticos o de clase. Esto te permitirá alcanzar un mayor grado de comprensión sobre el tema.

A diferencia de los métodos de instancia (MEMBER), que requieren que se cree una instancia del objeto para poder ser llamados, los **métodos estáticos se invocan directamente utilizando el nombre del tipo de dato objeto**. Esto significa que **no es necesario inicializar un objeto para acceder a un método estático**; basta con referirse al tipo de dato objeto seguido del nombre del método. Esta característica es especialmente **útil** en situaciones donde la **lógica implementada** en el método **no depende de los atributos** de ninguna instancia específica.

En la siguiente imagen se muestra un ejemplo de cómo se invocaría un método estático en un pequeño *script* PL/SQL:

```
DECLARE
    edad_jub NUMBER;
BEGIN
    -- Llamada al método estático utilizando
    -- el nombre del tipo de objeto
    edad_jub := Persona.edad_jubilacion();

    -- Mostrar el resultado en la consola
    DBMS_OUTPUT.PUT_LINE('Edad de jubilación: ' || edad_jub);
END;
/
```

Fig.14. Acceso a un método estático.

/ 8. Parámetro SELF

Todos los métodos que puede contener un tipo de dato objeto determinado aceptan como primer parámetro la instancia **SELF**, que es una **instancia predefinida** en Oracle del mismo tipo de dato objeto. Este **parámetro especial** se declara **automáticamente** en el sistema, aunque no se especifique explícitamente en la declaración del método, y siempre se crea como el **primer parámetro** que se pasa a un método.

En la siguiente imagen se muestra un **ejemplo** de una declaración explícita de este parámetro en la parte de especificación:

```
CREATE TYPE complejo AS OBJECT (
    MEMBER FUNCTION transformación (SELF IN OUT complejo)
…
);
```

Fig.15. Ejemplo de utilización de SELF.

Obviamente, el parámetro **SELF** tendrá asociado el mismo **tipo de dato** que el del objeto. Por otro lado, el **comportamiento** del parámetro **SELF** varía entre **funciones** y **procedimientos**:

- **En el caso de las funciones**, si el parámetro **SELF** no se declara de manera explícita, su modo por defecto será **IN**, lo que significa que solo se puede utilizar como un valor de entrada.
- **En los procedimientos**, si no se especifica claramente el parámetro **SELF**, su modo por defecto será **IN OUT**, lo que permite tanto la entrada como la modificación del valor durante la ejecución.

También es posible hacer referencia al parámetro **SELF** dentro del cuerpo de un método, especialmente cuando es necesario referirse al mismo objeto que está invocando el método. Con el ejemplo de la siguiente imagen se entenderá mejor:

```
MEMBER PROCEDURE setSalario (Salario NUMBER)
    IS BEGIN
    SELF.Salario := Salario;
END setSalario;
```

Fig.16. Ejemplo de uso de SELF (2).

La sentencia **SELF.Salario** hace referencia al atributo **Salario** del objeto que ha invocado el método, mientras que **Salario** se refiere al parámetro recibido por el método.

Este es un concepto que probablemente ya conozcamos, ya que es ampliamente utilizado en lenguajes de programación orientados a objetos, como **Java**.

/ 9. Fundamentos de la sobrecarga de métodos

La **sobrecarga de métodos** en programación orientada a objetos **permite definir varios métodos con el mismo nombre en una clase u objeto**, siempre que sus **parámetros difieran**. Esto mejora la legibilidad y reutilización del código, permitiendo operaciones similares con distintos tipos o cantidades de argumentos.

En lenguajes como Java y Oracle PL/SQL, la sobrecarga se utiliza para ofrecer flexibilidad en cómo los métodos son llamados e implementados. En Oracle PL/SQL, un método puede manejar distintas combinaciones de parámetros, facilitando la interacción con los objetos de manera más eficiente.

9.1. Condiciones y reglas para la sobrecarga

Para que la sobrecarga sea válida, deben cumplirse ciertas condiciones relacionadas con los parámetros del método:

- **Número de parámetros**: Los métodos **deben diferir en la cantidad de parámetros**. Por ejemplo, un método con un parámetro puede sobrecargarse con otro que tenga más.
- **Tipo de datos de los parámetros**: Los métodos **pueden diferir en el tipo de datos de los parámetros**. Un método que recibe un entero puede sobrecargarse con otro que reciba un texto o un número de coma flotante.
- **Orden de los parámetros**: La sobrecarga también es posible si los métodos tienen los mismos tipos de parámetros, pero en un orden diferente.

En la siguiente imagen se muestra un **ejemplo de sobrecarga** con el método calcular_area, donde, al recibir **un** único **parámetro**, este será **interpretado** como el **radio** para calcular el **área de un círculo**, y al recibir **dos parámetros**, serán tomados como los **lados** para calcular el **área de un rectángulo**.

```
MEMBER PROCEDURE calcular_area(radio NUMBER);
MEMBER PROCEDURE calcular_area(lado1 NUMBER, lado2 NUMBER);
```

Fig.17. Sobrecarga cálculo de áreas.

Es importante tener en cuenta las **limitaciones** de la sobrecarga:

- **Diferencias en nombres de parámetros**: Si los métodos solo difieren en los nombres de los parámetros, pero no en número, tipo u orden, no se permite la sobrecarga. Por ejemplo, un método con un parámetro llamado 'x' y otro con 'y' no pueden sobrecargarse si son del mismo tipo y número.
- **Tipo de retorno en funciones**: Las funciones no pueden sobrecargarse solo por su tipo de retorno. Si dos de ellas tienen el mismo nombre y parámetros, pero devuelven tipos de datos diferentes, la sobrecarga no es válida. En la siguiente imagen se muestra un ejemplo de este caso:

```
MEMBER FUNCTION obtener_datos(id_cliente NUMBER) RETURN VARCHAR2,
MEMBER FUNCTION obtener_datos(id_cliente NUMBER) RETURN DATE
```

Fig.18. Ejemplo de sobrecarga no válido.

10. Aplicación avanzada de la sobrecarga

Es importante tener en cuenta dos aspectos a la hora de aplicar la sobrecarga en el código. Por un lado, la **eficiencia** a la hora de ejecutarse y, por otro lado, el **mantenimiento del código**.

10.1. Optimización y eficiencia

La **sobrecarga** puede impactar en el **rendimiento**, especialmente si se usa en exceso. Es importante optimizar su uso para evitar que la resolución de métodos se vuelva un proceso costoso en sistemas de alto rendimiento.

En la siguiente imagen se muestra un ejemplo de sobrecargas que podría penalizar la eficiencia:

```
MEMBER PROCEDURE procesar_datos(p_id NUMBER),

MEMBER PROCEDURE procesar_datos(p_id NUMBER,
                                p_opciones VARCHAR2),

MEMBER PROCEDURE procesar_datos(p_id NUMBER,
                                p_opciones VARCHAR2,
                                p_flag BOOLEAN)
```

Fig.19. Optimización de sobrecarga.

En este ejemplo, cada procedimiento maneja diferentes niveles de complejidad. Sin embargo, un exceso de sobrecargas puede aumentar el tiempo de resolución de los métodos. Es recomendable limitar la sobrecarga a situaciones donde realmente aporte valor, considerando su impacto en el rendimiento, especialmente en métodos que se utilizan con frecuencia.

10.2. Diseño orientado al mantenimiento

Al diseñar sobrecargas, es importante pensar en la **mantenibilidad a largo plazo**. Un diseño coherente y bien documentado facilita futuras extensiones del código.

En la siguiente imagen se muestra un ejemplo:

```
MEMBER PROCEDURE actualizar_cliente(p_id_cliente NUMBER,
                                    p_direccion VARCHAR2),
MEMBER PROCEDURE actualizar_cliente(p_id_cliente NUMBER,
                                    p_direccion VARCHAR2,
                                    p_telefono VARCHAR2)
```

Fig.20. Diseño mantenible.

Si en el futuro se requiere agregar más parámetros, las sobrecargas actuales deben permitir fácilmente esa expansión sin necesidad de reescribir los métodos existentes.

Es importante mantener las sobrecargas consistentes y seguir un patrón lógico, lo que facilita su mantenimiento y futuras ampliaciones. Al hacerlo, es **necesario** encontrar un **equilibrio** entre la **optimización** del rendimiento y la facilidad de **mantenimiento**.

/ 11. Constructores

En Oracle, **todos los tipos de objetos vienen con un método predeterminado conocido como constructor**. Este es **responsable de crear nuevas instancias del objeto**, utilizando la especificación del tipo para inicializar sus atributos con valores específicos y devolviendo una instancia del objeto al sistema.

El **nombre del método constructor coincide con el del tipo de dato objeto**, y sus **parámetros se corresponden con los atributos** de dicho tipo. En la siguiente imagen, se muestra un ejemplo en el que se construyen dos instancias de los objetos 'dirección' y 'cliente':

```
dirección ('Calle Libertad', 'Getafe',
           'Madrid', '10', '1', 'A'),
cliente (35, 'Ana García Jiménez',
         dirección('Calle Oca, 'Madrid',
                   'Madrid', '3', '4', 'B'),
        683928121, 01/03/1982)
```

Fig.21. Ejemplo de uso de constructores.

Además del constructor por defecto, y al igual que en los lenguajes de programación orientados a objetos, **los usuarios pueden definir nuevos constructores**, permitiendo así la creación de un constructor con diferentes parámetros. Otra opción es **sustituir el constructor por defecto** utilizando la sentencia **CONSTRUCTOR FUNCTION**.

🔗 Enlaces de interés...

En el siguiente enlace, encontrarás un vídeo en el que podrás descubrir algunos ejemplos de definición de métodos constructores de los tipos de dato objeto en SQL Developer. Te permitirán ampliar tus conocimientos sobre el tema: https://www.youtube.com/watch?v=xSjxsnt_ilA

De este modo, **es posible definir múltiples constructores**, **siguiendo el concepto de sobrecarga de métodos** mencionado anteriormente. Es decir, se pueden tener diferentes constructores, cada uno con una lista distinta de parámetros.

En el ejemplo de la siguiente imagen, se declara un método constructor. En su cuerpo, se verifica si el cliente tiene un balance de compras negativo, en cuyo caso se inicializa a cero:

```
CONSTRUCTOR FUNCTION cliente (id_cliente NUMBER, compras NUMBER)
            RETURN SELF AS RESULT

CREATE OR REPLACE TYPE BODY cliente
    AS CONSTRUCTOR
    FUNCTION cliente(id_cliente NUMBER, compras NUMBER)
    RETURN SELF AS RESULT IS
BEGIN
    IF (compras >= 0) THEN
        SELF.compras := compras;
    ELSE
        SELF.compras := 0;
    END IF;
    RETURN;
END;
```

Fig.22. Ejemplo de creación de constructor.

/ 12. Caso práctico 1: "Creación de tipo de dato objeto en Oracle Express Database"

Planteamiento: Vamos a realizar un ejercicio básico de declaración de objetos en Oracle Express Database, siguiendo un proceso similar al mostrado en el vídeo de este tema.

Nudo: Vamos a crear un objeto básico llamado 'dirección', que tendrá los atributos 'calle', 'ciudad' y 'código postal'. Luego, definiremos un objeto 'persona', que incluirá los atributos 'nombre', 'apellidos', 'dirección' y 'fecha de nacimiento'. Finalmente, crearemos un tercer tipo de dato objeto llamado 'prueba', con los campos que queramos. Una vez hecho esto, utilizaremos el comando DROP TYPE para eliminarlo.

Desenlace: En las siguientes imágenes se muestran las instrucciones necesarias para crear estos dos tipos de objeto en Oracle SQL Developer:

```
create or replace TYPE direccion AS OBJECT(
    calle VARCHAR(50),
    ciudad VARCHAR2(50),
    codigo_postal NUMBER(5)
);
```

Fig.23. Creación del tipo de dato objeto dirección.

```
create or replace TYPE persona AS OBJECT(
    nombre VARCHAR(50),
    apellidos VARCHAR2(100),
    direccion VARCHAR2(100),
    fecha_nacimiento DATE
);
```

Fig.24. Creación del tipo de dato objeto persona.

/ 13. Caso práctico 2: "Definición de atributos y métodos para números complejos"

Planteamiento: En esta ocasión, aplicaremos la teoría revisada en los apartados anteriores del tema, declarando un tipo de datos que represente un número complejo e implementando tanto su especificación como su código.

Nudo: Vamos a realizar la declaración de un tipo de dato objeto 'complejo' que contiene dos atributos, 'parte real' y 'parte imaginaria' y dos funciones, 'aumento' y 'reducción'. La función 'aumento' implementará una suma de un número complejo que se le pasa por parámetro, tanto en la parte real como en la parte imaginaria. La función 'reducción' realizará el proceso contrario.

Desenlace: En la siguiente imagen se muestra un ejemplo del código necesario para esta especificación:

```
CREATE TYPE complejo AS OBJECT (
    parte_real NUMBER,
    parte_imaginaria NUMBER,
    MEMBER FUNCTION aumento (a IN complejo)
            RETURN complejo,
    MEMBER FUNCTION reduccion (a IN complejo)
            RETURN complejo
);
/
CREATE OR REPLACE TYPE BODY complejo AS
    MEMBER FUNCTION aumento (a IN complejo)
        RETURN complejo IS
    BEGIN
        RETURN complejo(parte_real + a.parte_real,
                        parte_imaginaria + a.parte_imaginaria);
    END aumento;

    MEMBER FUNCTION reduccion (a IN complejo)
        RETURN complejo IS
    BEGIN
        RETURN complejo(parte_real - a.parte_real,
                        parte_imaginaria - a.parte_imaginaria);
    END reduccion;
END;
/
```

Fig.25. Solución al caso práctico planteado.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema se ha introducido el concepto de bases de datos objeto-relacionales, cuyas características principales y tipos de dato objeto se han analizado en detalle, con especial enfoque en Oracle.

Además, se ha explicado la declaración de tipos de objeto, atributos, métodos (incluyendo métodos estáticos y constructores), sobrecarga de métodos y el parámetro SELF. Todos ellos son fundamentales para la comprensión y el uso de bases de datos objeto-relacionales, y servirán de base para temas más avanzados en la asignatura.

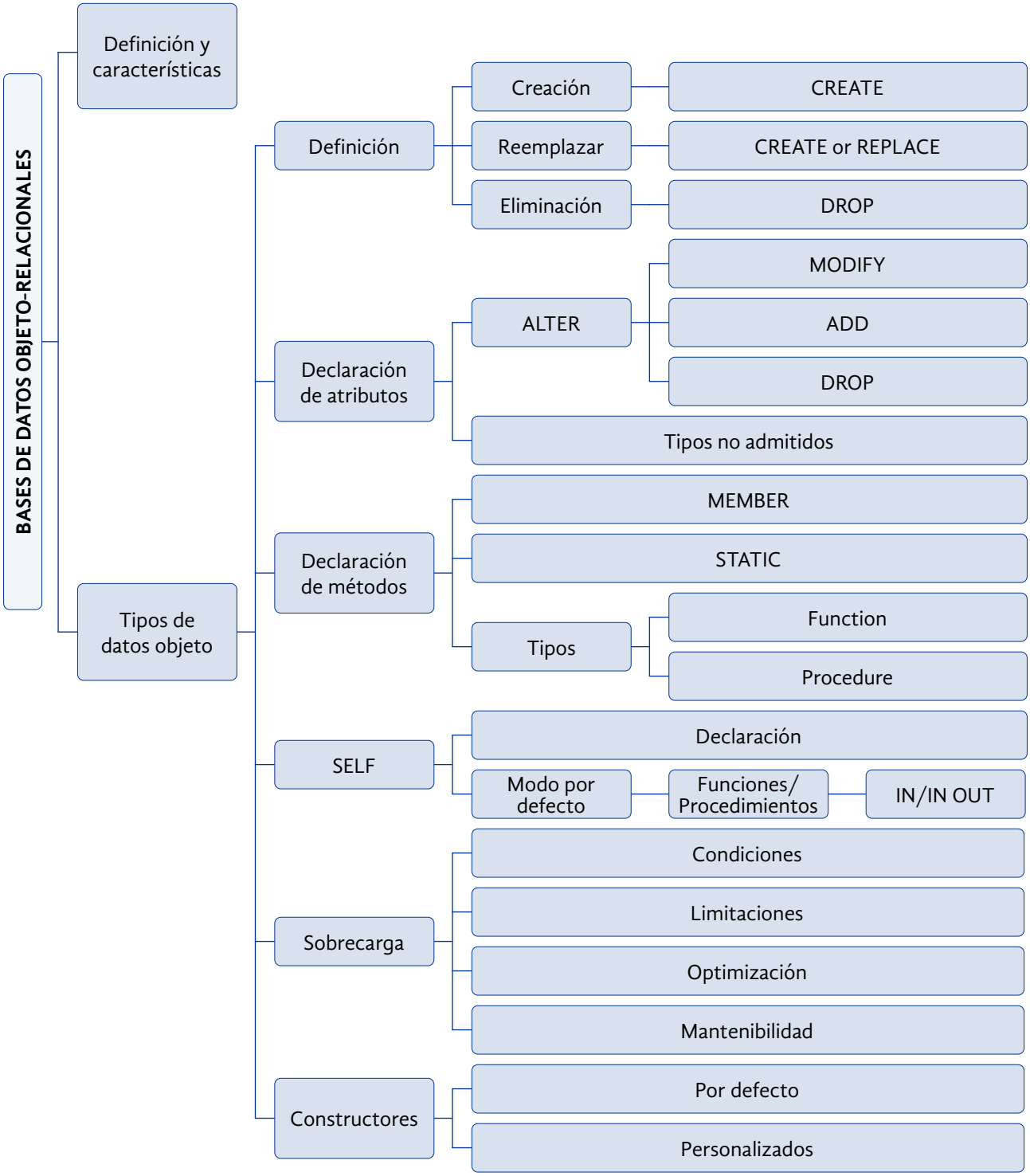


Fig.26. Esquema resumen del tema .

Resolución del caso práctico de la unidad

El proceso de modelar un coche en una base de datos objeto-relacional se puede abordar de diversas maneras. Una opción sería representarlo como un objeto que incluye varios atributos, tales como color, fabricante, ruedas, chasis, motor, entre otros. Algunos de estos atributos, como el motor, el chasis o las ruedas, podrían también ser representados como objetos independientes, cada uno con sus propios atributos, como fabricante, capacidad, potencia, etc.

En cuanto a los métodos, se podrían implementar varios, basados en los datos que proporcionan los vehículos modernos a través de su ordenador de a bordo. Estos métodos podrían incluir el cálculo del consumo instantáneo de combustible, el consumo medio, la emisora de radio sintonizada, la detección de averías, la presión de los neumáticos (que podría ser un método específico del objeto rueda) y otros similares.

Como se puede observar, las posibilidades son numerosas y el enfoque de modelado dependerá, en última instancia, del propósito que se desee lograr con el sistema. Sin embargo, lo que está claro es que este enfoque orientado a objetos nos permite integrar, en cada tipo de dato, los métodos necesarios para gestionar eficazmente la información que contiene.

/ 15. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, I. y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases. A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.